DETAILED TECHNICAL GUIDE

지도 · API 코드 설명서

TXT 설명을 반영한 상세판 - 구조, 문법, 실행 흐름

춘천 음식점 지도 프로젝트

카카오 지도 · Google 지도 · GeoJSON · GPS · 길찾기

1. 한눈에 보는 파일 역할

파일	담당 기능
restaurantMap.js	카카오 지도, 행정동 경계, 음식점 마커, 현재 위치
restaurantExplorer.js	지역 · 카테고리 · 즐겨찾기 필터와 목록 연동
directionsService.js	카카오모빌리티 REST API로 자동차 경로 요청
directionsController.js	출발지 · 도착지와 경로선 UI 제어
geolocationService.js	브라우저 GPS 현재 위치 조회 · 추적
nearbyRestaurantService.js	직선거리 계산과 주변 맛집 검색
restaurant_homepageMap.js	메인 홈페이지 Google 미니 지도

핵심 구분

맛집 탐색 화면은 카카오 지도, 메인 · 상세 미니 지도는 Google 지도를 사용한다.

2. 카카오 지도와 GeoJSON

춘천시 외곽선과 행정동 면 영역은 GeoJSON 파일에서 읽는다. 지도 화면은 카카오 JavaScript SDK가 담당한다.

1	2	3	4
GeoJSON 요청	JSON 객체 변환	좌표 순서 변환	Polygon · Polyline 생성

원본 위치: js/restaurantMap.js 164~171줄

```
const responses = await Promise.all([
  fetch(DISTRICT_URL),
  fetch(CITY_URL)
]);

if (responses.some(response => !response.ok)) {
  throw new Error('경계 파일을 불러오지 못했습니다.');
}

[districtGeoJson, cityGeoJson] = await Promise.all(
  responses.map(response => response.json())
);
```

왜 response.ok를 확인하나?

fetch는 404나 500 응답에서도 Promise 자체가 완료될 수 있다. 따라서 HTTP 요청이 성공 범위(200~299)인지 직접 확인한다.

response => !response.ok를 풀어 쓰면

```
// 화살표 함수
response => !response.ok

// 같은 뜻의 일반 함수
function (response) {
  return !response.ok;
}
```

responses.some(...)은 배열의 응답을 하나씩 검사한다. response.ok가 false인 응답을 찾으면 !false가 true가 되고, some()은 즉시 true를 반환한다. 즉 '두 파일 중 하나라도 실패했는가?'를 검사하는 코드다.

좌표 순서를 왜 바꾸나?

```
// GeoJSON: [경도, 위도]
// Kakao LatLng: (위도, 경도)
const latLng = coordinate =>
  new kakao.maps.LatLng(coordinate[1], coordinate[0]);
```

> ★ 예상 질문
> GeoJSON을 사용한 이유는? 행정동 이름뿐 아니라 실제 면 경계 좌표가 필요했기 때문이다.

Polygon과 MultiPolygon 처리

원본 위치: js/restaurantMap.js 46줄

```
const polygonsOf = geometry =>
  geometry.type === 'Polygon'
    ? [geometry.coordinates]
    : geometry.coordinates;
```

Polygon과 MultiPolygon은 좌표 배열의 중첩 깊이가 다르다. Polygon도 배열로 한 번 감싸면 이후 반복문이 두 형식을 같은 방식으로 처리할 수 있다.

경계 도형을 다시 그리는 이유

사용자가 행정동 선택을 바꾸면 과거 Polygon과 Polyline을 먼저 setMap(null)로 제거한다. 제거하지 않으면 이전 경계와 새 경계가 겹치고, 선택 상태와 화면이 달라진다.

```
const clearOverlays = overlays => {
  overlays.forEach(overlay => overlay.setMap(null));
  overlays.length = 0;
};
```

> 실행 결과
> '전체' 선택 시 춘천시 외곽선을, 특정 동 선택 시 행정동 면을 강조하고 LatLngBounds로 선택 영역 전체가 보이게 한다.

3. 음식점 마커와 필터

원본 위치: js/restaurantMap.js 203~228줄

새 데이터를 그리기 전에 기존 마커를 지도에서 제거하고 배열도 비운다. 그렇지 않으면 실행할 때마다 같은 마커와 클릭 이벤트가 겹친다.

```
restaurantMarkers.forEach(({ marker }) =>
  marker.setMap(null)
);
restaurantMarkers = [];
```

> 정확한 의미
> 카카오 지도 전체를 초기화하는 것이 아니라 음식점 마커만 정리한 뒤 최신 데이터로 다시 만든다.

같은 좌표의 음식점 묶기

동일 건물에 음식점이 여러 개 있으면 마커가 완전히 겹친다. 위도와 경도를 소수점 8자리 문자열로 만든 뒤 Map의 key로 사용해 같은 위치를 한 그룹으로 묶는다.

```
const groups = new Map();

restaurants.forEach(restaurant => {
  const key =
    `${restaurant.latitude.toFixed(8)},` +
    `${restaurant.longitude.toFixed(8)}`;
  const group = groups.get(key) || [];
  group.push(restaurant);
  groups.set(key, group);
});
```

마커는 그룹마다 하나만 생성하고, 클릭 시 같은 위치의 음식점 목록을 정보창에서 보여준다. toFixed(8)은 작은 부동소수점 차이 때문에 같은 위치가 다른 그룹이 되는 문제를 줄인다.

필터 조건

```
const matchesFilter = restaurant =>
  (selectedDistricts.has(ALL) ||
    selectedDistricts.has(restaurant.district)) &&
  (selectedCategory === ALL ||
    restaurant.category === selectedCategory) &&
  (favoriteRestaurantKeys === null ||
    favoriteRestaurantKeys.has(getRestaurantKey(restaurant)));
```

지역 · 카테고리 · 즐겨찾기 조건을 모두 통과한 음식점만 표시한다. `marker.setMap(map)`은 표시, `marker.setMap(null)`은 숨김이다.

즐겨찾기 데이터 연결

원본 위치: js/restaurantExplorer.js 16~38줄

즐겨찾기 저장 ID는 '행정동_카테고리_순번'이지만 지도 CSV는 그 순번을 직접 알지 못한다. 따라서 즐겨찾기 항목을 '행정동|음식점명' 형태로 변환해 지도 데이터와 비교한다.

```
function syncFavoriteMarkers() {
  window.RestaurantMap?.selectFavorites(
    favoritesOnly
      ? getFavoriteRestaurantKeys()
```

```
        : null
    );
}
```

favoritesOnly가 true면 즐겨찾기 키 배열을 전달하고, false면 null을 전달해 필터를 해제한다. ?.는 RestaurantMap이 아직 준비되지 않은 경우 호출 오류를 막는다.

> ★ 예상 질문
> 같은 좌표의 음식점은 왜 묶나? 같은 건물의 마커가 겹치는 문제를 막고 하나의 정보창에서 여러 음식점을 보여주기 위해서다.

4. GPS와 주변 맛집

1	2	3	4	5
위치 권한 요청	현재 좌표 획득	직선거리 계산	반경 필터	거리 표시

원본 위치: js/geolocationService.js 24~40줄

```
navigator.geolocation.getCurrentPosition(
  position => resolve({
    latitude: position.coords.latitude,
    longitude: position.coords.longitude,
    accuracy: position.coords.accuracy
  }),
  error => reject(new Error(getErrorMessage(error)))
);
```

GPS 옵션과 오류 처리

옵션	의미	고려할 점
enableHighAccuracy	가능하면 높은 정확도 요청	응답·배터리 비용 증가 가능
timeout: 10000	최대 10초 대기	초과 시 오류 코드 3
maximumAge: 0	캐시 위치 사용 안 함	새 위치를 요청

오류 코드 1은 권한 거부, 2는 위치 확인 불가, 3은 시간 초과다. 숫자 코드를 사용자에게 이해되는 문장으로 변환한다. 페이지를 나갈 때 clearWatch로 위치 추적을 중지해 불필요한 조회를 막는다.

하버사인 공식이 필요한 이유

위도·경도는 평면 좌표가 아니므로 단순 차이만으로 km 거리를 구할 수 없다. 하버사인은 지구 곡면을 고려하고, 음식점마다 길찾기 API를 호출하지 않아 주변 후보를 빠르게 찾는다.

원본 위치: js/nearbyRestaurantService.js 30~50줄

```
const haversine =
  Math.sin(latitudeDifference / 2) ** 2 +
  Math.cos(startLatitude) *
  Math.cos(endLatitude) *
  Math.sin(longitudeDifference / 2) ** 2;

return EARTH_RADIUS_KM * 2 * Math.atan2(
  Math.sqrt(haversine),
```

```
    Math.sqrt(1 - haversine)
  );
```

반경 필터와 정렬

각 음식점까지 distanceKm를 계산한 뒤 radiusKm보다 먼 항목은 제외한다. sortBy가 distance이면 가까운 순, rating이면 평점·리뷰 수를 우선하고 동률일 때 거리순으로 정렬한다. 좌표가 잘못된 음식점은 개별 try/catch로 제외해 전체 검색이 중단되지 않게 한다.

> 실제 화면
> 메인 홈페이지의 '내 주변 맛집' 탭에서 250m · 500m · 1km · 직접 설정 반경과 음식점 거리를 표시한다.

> 주의
> 하버사인 결과는 직선거리다. 실제 자동차 도로 거리는 카카오 길찾기 API 결과를 사용한다.

5. 카카오 자동차 길찾기

1	2	3	4	5
출발지 · 도착지	REST API 요청	JSON 응답	vertexes 추출	Polyline 표시

원본 위치: js/directionsService.js 15~64줄

```
  const response = await fetch(
    `${DIRECTIONS_ENDPOINT}?${query}`,
    {
      headers: {
        Authorization: `KakaoAK ${getApiKey()}`
      }
    }
  );

  if (!response.ok) {
    throw new Error(`길찾기 요청 실패 (${response.status})`);
  }

  const data = await response.json();
```

요청 query의 의미

항목	값	설명
origin	경도,위도	자동차 출발 위치
destination	경도,위도	자동차 도착 위치
priority	RECOMMEND	추천 경로 요청
alternatives	false	대안 경로 생략
road_details	false	상세 도로 정보 생략

지도 LatLng는 위도,경도 순서지만 REST API query는 경도,위도 순서다. 두 규격을 혼동하면 다른 위치의 경로가 검색되므로 요청 단계와 표시 단계를 구분해야 한다.

두 단계의 오류 검사

첫째 response.ok로 HTTP 통신 성공을 확인한다. 401·403은 인증 또는 권한 문제로 따로 안내한다. 둘째 route.result_code로 실제 자동차 경로 계산 성공을 확인한다. HTTP가 200이어도 경로가 없을 수 있기 때문이다.

경로 좌표 구조

응답 구조는 routes → sections → roads → vertexes다. vertexes는 [경도, 위도, 경도, 위도, ...] 형태라 두 칸씩 읽는다.

원본 위치: js/directionsController.js 146~164줄

```
const path = route.points.map(point =>
  new kakao.maps.LatLng(
    point.latitude,
    point.longitude
  )
);

routePolyline = new kakao.maps.Polyline({
  map, path,
  strokeWeight: 7,
  strokeColor: '#eb1111'
});
```

> ★ 예상 질문
> JavaScript SDK와 REST API의 차이는? SDK는 지도 요소를 브라우저에 그리며, REST API는 서버가 계산한 자동차 경로 JSON을 반환한다.

화면 제어와 중복 요청 방지

길찾기 버튼을 누르면 출발지·도착지 존재 여부를 먼저 검사한다. 요청 중에는 버튼을 disabled 상태로 만들고, try에서 요청·표시, catch에서 오류 안내, finally에서 버튼 복구를 수행한다.

```
try {
  const route = await DirectionsService.findDrivingRoute(
    routeLocations.origin,
    routeLocations.destination
  );
  drawRoute(route);
} catch (error) {
  clearRoute();
  showRouteResult(error.message, true);
} finally {
  routeSubmitButton.disabled = false;
}
```

6. 지도 API를 불러오는 순서

카카오 지도 SDK

1	2	3	4	5
설정 파일에서 키 읽기	script 요소 생성	SDK 주소 지정	head에 추가	onload 후 지도 생성

원본 위치: js/restaurantMap.js 257줄부터

```
const script = document.createElement('script');
script.src =
```

```
    'https://dapi.kakao.com/v2/maps/sdk.js' +
    `?appkey=${encodeURIComponent(apiKey)}` +
    '&autoload=false';
  script.async = true;
  script.onload = () => kakao.maps.load(initFoodMap);
  document.head.appendChild(script);
```

createElement만으로는 다운로드가 시작되지 않는다. appendChild로 실제 HTML head에 넣는 순간 브라우저가 SDK를 요청한다.

onload, onerror, autoload=false

script.onload는 SDK 파일 다운로드 성공 후 실행되고, script.onerror는 네트워크 문제 등으로 파일 자체를 받지 못했을 때 실행된다. autoload=false를 사용했으므로 onload 이후 kakao.maps.load(initFoodMap)을 직접 호출해 SDK 내부 준비가 끝난 뒤 지도를 생성한다.

설정 파일이 먼저 실행되어야 하는 이유

```
<!-- 키를 먼저 window에 등록 -->
<script src="../js/map-config-KakaoJS.js"></script>

<!-- 그 다음 키를 읽는 지도 코드 실행 -->
<script src="../js/restaurantMap.js"></script>
```

HTML의 일반 script 태그는 위에서 아래로 실행된다. 순서를 바꾸면 restaurantMap.js가 실행될 때 window.KAKAO_JS_MAPS_API_KEY가 없어 지도가 비활성화될 수 있다.

Google 미니 지도 SDK

Google 지도는 URL의 callback 이름을 SDK 로딩 완료 후 호출한다. 따라서 초기화 함수를 window에 등록해야 한다.

```
  window.initHomepageMap = initHomepageMap;
  script.src =
    'https://maps.googleapis.com/maps/api/js' +
    `?key=${encodeURIComponent(apiKey)}` +
    '&callback=initHomepageMap' +
    '&language=ko&region=KR';
```

7. API 키 보안과 구조 개선

> 중요
> 브라우저 JavaScript에 들어간 키는 완전히 숨길 수 없다. 지도 JavaScript 키에는 허용 도메인과 API 제한을 반드시 설정한다.

카카오 REST 키는 프런트엔드 코드와 네트워크 요청에서 노출될 수 있다. 실제 배포에서는 백엔드가 키를 환경 변수로 보관하고 카카오 API를 대신 호출하는 구조가 더 안전하다.

이미 공개 저장소에 올라간 키는 파일에서 지우는 것만으로 부족하다. 키를 폐기·재발급하고 Git 기록 노출 여부도 확인한다.

권장 구조

1	2	3	4
브라우저	자체 백엔드	카카오 API	필요 결과만 반환

브라우저는 자체 백엔드에 출발지·도착지만 보낸다. 백엔드는 환경 변수에 저장한 REST 키로 카카오 API를 호출한다. 이렇게 하면 REST 키가 브라우저 코드와 개발자 도구에 직접 노출되지 않는다.

추가 개선 항목

음식점 이름 대신 고유 ID 사용, 지도 제공자 통일, 마커 클러스터링, 화면 범위 기반 로딩, 요청 취소(AbortController), 오류 재시도 UI와 자동화 테스트를 적용할 수 있다.

8. 전체 작동 흐름

1 페이지 실행	2 지도 SDK 로딩	3 GeoJSON · CSV 로딩	4 경계 · 마커 표시

1 사용자 필터	2 마커 표시 갱신	3 GPS 선택	4 주변 맛집 계산

1 출발지 · 도착지	2 REST API	3 경로 좌표	4 지도 경로선

발표 전 빠른 체크

질문	한 문장 답변
메인 지도 파일	restaurant_homepageMap.js - Google 미니 지도
카카오 지도 파일	restaurantMap.js - 탐색 지도와 마커 · 경계
주변 맛집 거리	하버사인 직선거리 - API 요청 없이 후보 검색
자동차 경로	카카오모빌리티 REST API - 실제 도로 거리
가장 중요한 보안 개선	REST 키와 API 호출을 백엔드로 이동

설명 원본은 txt/map_code, 예상 질문 원본은 txt/Q&Atxt;에 보관되어 있다.